

Mini Debugger – User Documentation

Chirag Suthar
2025MCS2105

Gandem Nitin D.
2025MCS2152

1 Problem Statement

Design and implement a minimal UNIX-style debugger capable of controlling and inspecting a running program using operating system-level debugging primitives, without relying on existing debuggers such as `gdb`.

2 Introduction

The Mini Debugger is a lightweight command-line debugger built for UNIX-like systems. It demonstrates fundamental operating system concepts such as **process tracing, execution control, signal handling, and software breakpoints** using the `ptrace()` system call.

The debugger allows users to start a program under debugger control, pause and resume execution, set breakpoints at specific memory addresses, single-step instructions, and observe program termination behavior.

3 Starting the Debugger

3.1 Compilation

```
make
```

3.2 Running the Debugger

```
./mydbg
```

3.3 Prompt Example

```
mydbg>
```

4 Feature Overview

The Mini Debugger supports the following features:

- Launching a program under debugger control
- Continuing program execution
- Single-step instruction execution
- Software breakpoints using INT3 (0xCC)
- Breakpoint removal with safe instruction restoration
- CPU register inspection
- Process state reporting

- Safe handling of invalid commands
- Graceful program termination handling

5 Running a Program

Command: run

Usage

```
run <program>
```

Example

```
run ./test
```

Notes

- The program must exist and be executable
- Only one program can be debugged at a time
- Running a program initializes a fresh breakpoint table
- Argument parsing is minimal; only the program path is supported

6 Continuing Program Execution

Command: continue

```
continue
```

Execution resumes until a breakpoint is hit or the program exits.

7 Single-Step Execution

Command: step

```
step
```

8 Breakpoints

8.1 Setting a Breakpoint

Command: break

```
break <hexaddr>
```

8.2 Removing a Breakpoint

Command: delete

```
delete <hexaddr>
```

9 Register Inspection

Command: `regs`

```
regs
regs -s
regs -g
```

Registers are retrieved using `ptrace(PTRACE_GETREGS)`.

10 Process State Reporting

Command: `status`

Possible states:

- `NOT_STARTED`
- `RUNNING`
- `STOPPED`
- `EXITED`

11 Program Exit Handling

```
[dbg] child exited with status 0
```

12 Limitations

- No source-level debugging
- No symbol resolution
- No multi-threaded debugging
- No memory inspection

13 Exiting the Debugger

Command: `quit`

```
quit
```

14 Conclusion

The Mini Debugger provides a clean and educational demonstration of OS-level debugging using `ptrace`, forming a strong foundation for understanding real-world debuggers.